Cognitive Ability and Skill Change in Software Engineers in the Pre-AI and AI Assistant Era

Chevon Phillip

March 31, 2026

Abstract

This paper reviews available empirical evidence on whether software engineers who programmed without AI coding assistants or large language models experienced a measurable decline in cognitive ability over the last decade, and whether AI dependence is reducing human intelligence in specialized and critical work. The current research record does not provide direct longitudinal evidence of decade-scale cognitive decline among professional software engineers who did not use AI assistants. Instead, the strongest evidence comes from short-horizon comparative studies of AI-assisted and non-assisted programming, along with adjacent automation, cognitive offloading, and trust-calibration literatures. The evidence suggests heterogeneous productivity effects, clearer risks for learning and debugging skill formation, and stronger support for claims about skill redistribution, cognitive offloading, and automation-related degradation than for claims about a generalized decline in intelligence.

1. Executive Summary

- There is **no direct evidence** in the accessible research record showing a decade-long cognitive decline among professional software engineers who continued programming *without* AI assistants.
- Existing evidence is mainly based on short-term studies comparing **AI-assisted** and **non-assisted** programming conditions.
- Short-term productivity findings are **mixed**: some studies show faster completion with AI assistance, while others show slower performance in realistic, high-context work.
- The clearest risk signal is not a drop in global intelligence, but reduced **learning, debugging practice, calibration, and independent problem solving** when AI substitutes for reasoning.
- The strongest theoretical framing comes from automation research, especially the concepts of **out-of-the-loop performance**, **skill degradation**, **cognitive offloading**, and **trust miscalibration**.
- The most defensible conclusion is that AI may shift work from first-principles generation toward **verification, review, integration, and correction**, with benefits or harms depending on how users engage with the tool.

2. Research Scope and Framing

This review covers research from roughly 2016 through early 2026 related to software engineering cognition, AI-assisted programming, and adjacent human-factors and cognitive-psychology literature. Older foundational sources were included where they directly explain mechanisms relevant to the question, especially automation-induced skill degradation and cognitive offloading.

The central questions were:

1. Has there been any demonstrated decline in cognitive ability among software engineers who have been programming without AI coding assistants or LLMs over the last 10 years?
2. What research has tested, demonstrated, or confirmed such a decline?
3. Is AI reducing human intelligence as people increasingly depend on it for specialized and critical tasks?

The answer to the first two questions is limited by the current evidence base: the research needed to make a strong claim would require long-term longitudinal measurement of developers' cognition, programming performance, and tool exposure. That evidence is largely absent.

3. What "Cognitive Ability" Means in Programming

In software engineering, cognitive ability is not one single measurable trait. Research usually operationalizes it through narrower constructs tied to programming work.

Table 1: Core cognitive dimensions relevant to programming research

Construct	**Meaning in programming**	**Typical operationalization**
Working memory	Holding intermediate states, constraints, and hypotheses while reading or debugging code	Standard working-memory tasks, comprehension load proxies, physiological or neurocognitive correlates
Problem solving	Decomposing requirements and searching the solution space	Timed coding tasks, correctness, strategy analyses
Debugging skill	Identifying, localizing, and fixing faults	Debugging quizzes, bug-fix tasks, error localization measures
Code comprehension	Building an internal model of code behavior across functions and modules	Code reading tasks, modification tasks, comprehension questions, eye-tracking or neural measures
Creativity	Producing novel abstractions or solution paths	Rarely measured directly in professional programming studies; sometimes inferred from solution diversity
Metacognition and calibration	Knowing what one knows and when to verify, doubt, or intervene	Confidence vs. accuracy, trust and reliance measures, verification behavior

A key issue is that most programming studies do *not* measure changes in broad intelligence. They usually measure task performance, learning, comprehension, or debugging accuracy.

4. Evidence About Decline Among Non-AI Programmers Over the Last Decade

4.1. What Strong Evidence Would Require

To support a claim that programmers who did not use AI assistants experienced cognitive decline over 10 years, the literature would need something like the following:

- a longitudinal cohort of professional developers measured repeatedly over many years,
- stable instruments for reasoning, debugging, comprehension, or related cognitive performance,
- explicit logging of tool exposure over time, including sustained non-AI conditions,
- controls for aging, role drift, experience, programming language changes, and workflow shifts.

4.2. What the Current Literature Actually Shows

The currently available literature does **not** provide that type of evidence. Instead, it mainly contains:

- cross-sectional or short-horizon programming cognition studies,
- recent AI-vs-no-AI experiments,
- systematic reviews showing that most GenAI-for-code literature is very recent,
- adjacent automation and cognitive offloading work outside software engineering.

4.3. Conclusion on the Non-AI Decline Question

The best-supported conclusion is:

> **There is currently insufficient empirical evidence to claim that software engineers who have been programming without AI assistants experienced a measurable decline in cognitive ability over the last 10 years.**

This is not proof that no such decline exists. It is a statement that the necessary longitudinal evidence has not been established.

5. Comparative Evidence: AI-Assisted vs. Non-Assisted Programming

Selected empirical results on AI coding assistance and performance (2016 to 2026)

- **2016:** Automation literature continues to emphasize out-of-the-loop and skill-degradation risks in high-automation settings.

- **2023:** A controlled Copilot trial reports substantial speed gains in a standardized task.
- **2024:** An in-IDE LLM tool for code understanding shows higher task progress versus web search, but no clear understanding gains.
- **2025:** An RCT on experienced open-source maintainers reports that AI slows completion time and shifts time toward prompting and review.
- **2025:** Survey evidence in knowledge work links higher confidence in GenAI to reduced critical-thinking effort and increased reliance.
- **2026:** An RCT on learning a new async library reports reduced quiz mastery for AI-assisted participants.

This is where the evidence is strongest. Several studies explicitly compare AI-assisted conditions with non-assisted conditions.

Table 2: Selected empirical studies comparing AI-assisted and non-assisted programming

Study	**Population**	**Exposure**	**Design**	**Measures**	**Main finding**
Peng et al. (2023)	95 professional programmers	Copilot vs. no Copilot	Controlled experiment	Completion time, success rate	AI group completed the standardized task 55.8% faster
Becker et al. / METR (2025)	16 experienced open-source developers, 246 issues	AI allowed vs. AI disallowed	Randomized controlled trial on real repository tasks	Completion time, activity allocation, forecasts vs. outcomes	AI increased completion time by 19% and users overestimated speedups
Shen and Tamkin / Anthropic (2026)	52 mostly junior software engineers learning Trio	AI-assisted learning vs. no AI during learning	Randomized experiment	Task time, post-task quiz, debugging, conceptual understanding	AI-assisted participants scored 17% lower on post-task mastery measures
Borg et al. (2025)	151 participants, mostly professionals	AI-assisted feature addition vs. no AI; later downstream evolution without AI	Two-phase controlled study	Completion time, downstream evolution quality	Phase 1 speed gains; no large downstream maintainability differences detected

Study	Population	Exposure	Design	Measures	Main finding
Nam et al. (2024)	32 mixed participants	In-IDE LLM support vs. web search	Within-subjects user study	Progress, completion, understanding	Better progress with AI support but no clear understanding gains
Kazemitabaar et al. (2023)	69 novice learners ages 10–17	AI code generation vs. baseline	Multi-session controlled experiment	Authoring performance, post-tests, retention	Higher authoring performance with mixed retention effects depending on learner background

5.1. Interpretation of the Comparative Evidence

The key pattern is heterogeneity:

- AI can improve speed in bounded, lower-context, or more standardized tasks.
- AI can slow experienced developers in realistic tasks when the review, correction, and integration burden dominates.
- AI use during learning may reduce mastery, especially in debugging and conceptual understanding.

6. Productivity Is Not the Same as Cognition

A major limitation across this research area is that productivity metrics are often used as a proxy for deeper cognition. This is not sufficient.

Task completion time can improve while underlying comprehension declines. Similarly, a developer may produce code faster but retain less conceptual knowledge or be less able to debug independently later.

Table 3: Selected time and mastery outcomes from AI programming studies

Study	Reported outcome
Peng et al. (2023)	55.8% faster completion in a standardized programming task
Becker et al. / METR (2025)	19% slower completion in real repository maintenance tasks
Shen and Tamkin (2026)	No significant main time improvement, but 17% lower post-task mastery
Nam et al. (2024)	Better task progress with no clear understanding improvement

7. Mechanisms: How AI Could Affect Human Cognition Without Reducing "Intelligence" Globally

The most useful way to frame the question is mechanistically. Rather than asking whether AI lowers intelligence in general, it is better to ask how AI changes cognitive work.

7.1. Out-of-the-Loop Performance and Skill Degradation

Automation research has long shown that higher automation can move humans from active control into monitoring roles. This can produce:

- reduced situation awareness,
- poorer takeover performance when automation fails,
- skill atrophy when manual and diagnostic practice decline,
- complacency or overreliance when trust exceeds true system capability.

In software engineering, the analogous concern is reduced practice in:

- generating code from first principles,
- debugging without assistance,
- maintaining detailed mental models of codebases,
- detecting subtle security or correctness failures in plausible-looking output.

7.2. Cognitive Offloading

Cognitive offloading refers to shifting internal cognitive work to external tools. In the context of AI-assisted programming, this may include:

- offloading recall of APIs and patterns,
- offloading intermediate reasoning during debugging,
- offloading evaluation effort by trusting fluent output too quickly.

Offloading is not necessarily harmful. It can be adaptive if it frees capacity for higher-order work. But it may harm learning if it removes desirable difficulty, retrieval practice, and repeated diagnostic effort.

7.3. Trust Calibration

The problem is not simply reliance, but **miscalibrated reliance**. When users trust AI output more than it deserves, they may verify less carefully, miss defects, or feel falsely confident. Several studies and surveys point to this risk.

8. Figure 1: Timeline of Major Empirical Signals

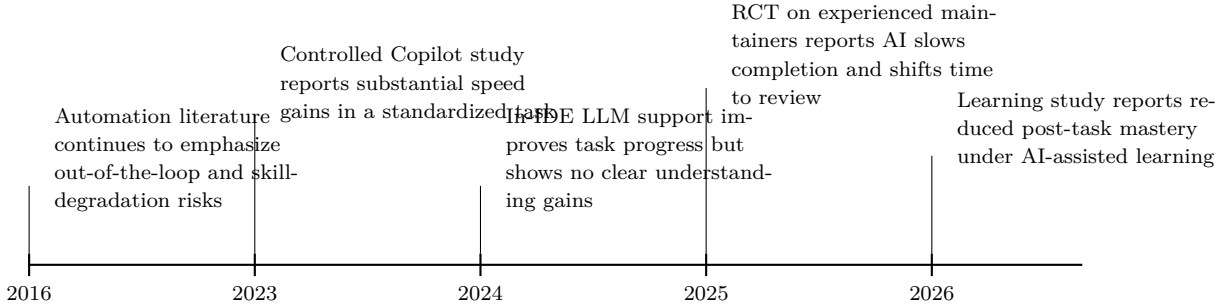

Figure 1: Selected empirical milestones relevant to cognition, productivity, and AI-assisted programming.

9. Figure 2: Conceptual Causal Pathways

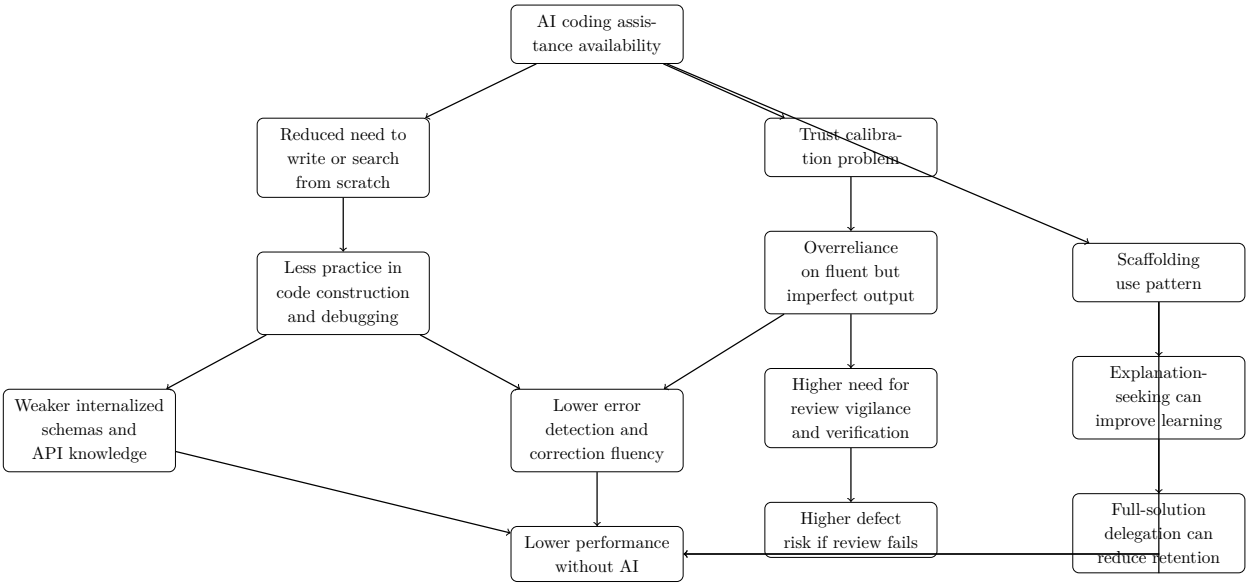

Figure 2: Conceptual model of how AI assistance may affect programming skill formation, verification burden, and dependence. This is a hypothesis diagram, not a claim that every pathway has been fully established causally.

10. Does AI Reduce Human Intelligence?

The current evidence does **not** justify the broad claim that AI is reducing human intelligence in a global or IQ-like sense. That claim is too strong for the evidence available.

What the evidence *does* support is narrower and more defensible:

- AI can reduce independent practice in reasoning-heavy tasks.

- AI can weaken learning and mastery in some contexts, especially when used as a substitute rather than a scaffold.
- AI can produce overconfidence, trust miscalibration, and reliance behaviors that leave users less effective when the system is wrong.
- The resulting problem is better described as **skill redistribution or skill atrophy in specific domains**, not a demonstrated collapse of overall human intelligence.

11. Methodological Gaps

Several major gaps prevent stronger claims:

- lack of decade-scale longitudinal studies in professional developers,
- overreliance on short-horizon productivity measures,
- rapid AI model and tool churn,
- confounding from aging, role changes, programming language evolution, and workflow changes,
- self-report biases and miscalibrated perceptions of productivity.

12. Practical Implications

Even without perfect long-term science, organizations can monitor potential skill atrophy through:

- regular no-assistant debugging and comprehension drills,
- measuring review burden for AI-assisted code,
- confidence-vs-correctness calibration checks,
- preserving explanation-seeking and first-principles problem solving in training,
- avoiding shallow productivity proxies as the main measure of developer value.

13. Conclusion

The current empirical record does not support a confirmed decade-long decline in cognitive ability among software engineers who programmed without AI assistants. That specific claim has not been demonstrated because the necessary longitudinal evidence does not yet exist.

However, there is meaningful evidence that AI assistance can affect learning, debugging fluency, verification effort, and reliance patterns. The strongest concern is not that AI is directly lowering general human intelligence, but that it may reduce independent competence in specialized tasks when people rely on it in place of deliberate reasoning and practice.

The most accurate current conclusion is this: AI is not yet proven to be reducing human intelligence broadly, but it can plausibly erode specific forms of expertise and critical thinking under conditions of overreliance, weak verification, and reduced practice.

References

1. Bainbridge, L. (1983). *Ironies of automation.* Automatica, 19(6), 775–779. Available at: https://ckrybus.com/static/papers/Bainbridge_1983_Automatica.pdf

2. Becker, J., et al. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity.* arXiv. Available at: https://arxiv.org/pdf/2507.09089

3. Borg, M., et al. (2025). *Does code developed with AI coding assistants degrade software maintainability?* arXiv. Available at: https://arxiv.org/pdf/2507.00788

4. Chong, C. J., et al. (2024). *Assessing the quality of LLM-generated code for security and correctness.* arXiv. Available at: https://arxiv.org/pdf/2409.19182

5. Fitts, P. M. (1951). *Human engineering for an effective air-navigation and traffic-control system.* Relevance cited through later automation literature.

6. Lee, J. D., & See, K. A. (2004). *Trust in automation: Designing for appropriate reliance.* Human Factors, 46(1), 50–80. Available at: https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392

7. Nam, D., et al. (2024). *GILT: Helping developers understand unfamiliar code with in-IDE LLM support.* arXiv. Available at: https://arxiv.org/pdf/2307.08177

8. Parasuraman, R., & Riley, V. (1997). *Humans and automation: Use, misuse, disuse, abuse.* Human Factors, 39(2), 230–253. Available at: https://journals.sagepub.com/doi/10.1518/001872097778543886

9. Peng, S., et al. (2023). *The impact of AI on developer productivity: Evidence from GitHub Copilot.* arXiv. Available at: https://arxiv.org/pdf/2302.06590

10. Risko, E. F., & Gilbert, S. J. (2016). *Cognitive offloading.* Trends in Cognitive Sciences, 20(9), 676–688. Available at: https://pubmed.ncbi.nlm.nih.gov/27542527/

11. Shen, J. H., & Tamkin, A. (2026). *AI assistance during coding can reduce skill formation.* Anthropic Research / arXiv. Available at: https://www.anthropic.com/research/AI-assistance-coding-skills and https://arxiv.org/pdf/2601.20245

12. Sparrow, B., Liu, J., & Wegner, D. M. (2011). *Google effects on memory: Cognitive consequences of having information at our fingertips.* Science, 333(6043), 776–778. Available at: https://www.science.org/doi/abs/10.1126/science.1207745

13. GitClear. (2025). *AI copilot code quality report.* Available at: https://gitclear-public.s3.us-west-2.amazonaws.com/GitClear-AI-Copilot-Code-Quality-2025.pdf

14. Kazemitabaar, M., et al. (2023). *Studying the effect of AI code generators on novice learners.* Available at: https://www.terpconnect.umd.edu/~weintrop/papers/Kaze\protect\discretionary{\char\hyphenchar\font}{}{}mitabaar_et_al_CHI_2023.pdf

15. Microsoft Research. (2025). *The impact of generative AI on critical thinking: Self-reported reductions in independent effort under high confidence*. Available at: https://www.microsoft.com/en-us/research/wp-content/uploads/2025/01/lee_2025_ai_critical_thinking_survey.pdf

16. Peitek, N., et al. (2020). *A large-scale study of code comprehension and brain activity*. eLife, 9, e59340. Available at: https://elifesciences.org/articles/59340

17. Siegmund, J., et al. (2014/2020 related literature). *Measuring neural efficiency of program comprehension and expertise*. See also: https://pmc.ncbi.nlm.nih.gov/articles/PMC7877476/

18. Nature / Scientific Reports sources on programming cognition and expertise relevant to problem solving and comprehension. Available at: https://www.nature.com/articles/s41598-020-70360-z and https://www.nature.com/articles/s41598-024-56090-6

19. Systematic review of LLMs in code comprehension and developer assistance (2025). Available at: https://arxiv.org/html/2510.17894v2

20. Human factors background on automation-related performance degradation in high-automation systems. See also: https://journals.sagepub.com/doi/10.1518/001872095779064555